

Panoramica degli Tipi Astratti di Dato

(ADT Opachi v5.1)

Il sistema adotta una scomposizione modulare rigida basata sullo standard C99. Ogni Tipo Astratto di Dato (ADT) è esposto all'esterno unicamente tramite un **Puntatore Opaco** (*Opaque Pointer*), nascondendo la definizione fisica delle strutture concrete all'interno dei file `.c`. Questo garantisce il principio dell'**Information Hiding**, impedendo alterazioni maliziose della memoria da parte dell'interfaccia utente.

1. ADT ReportList (Linked List Dinamica)

- **File Header:** `include/adt/report_list.h`
 - **File Sorgente:** `src/adt/report_list.c`
 - **Definizione Fisica:** Struttura concatenata semplice con puntatore di controllo alla testa (`head`), iteratore interno per i cicli (`current`) e variabile pre-calcolata di dimensione (`size`).
 - **Operazioni Chiave:**
 - `list_insert(ReportList l, Report r)` : Inserimento in testa in tempo costante $O(1)$.
 - `list_remove(ReportList l, int report_id)` : Ricerca ed estrazione fisica del nodo in $O(n)$, con decremento atomico della `size`.
 - `list_rewind(ReportList l)` / `list_next(ReportList l)` : Logica nativa dell'iteratore per scorrere sequenzialmente i record in RAM senza esporre i nodi della catena.
-

2. ADT ReportStack (Undo Buffer LIFO)

- **File Header:** `include/adt/report_stack.h`
- **File Sorgente:** `src/adt/report_stack.c`
- **Definizione Fisica:** Array statico preallocato limitato a un massimo di 10 slot gestito tramite un indice di cima (`top`).
- **Operazioni Chiave:**
 - `stack_push(ReportStack s, Report r)` : Esegue una clonazione profonda (*deep copy*) dell'oggetto Report originario per metterlo al sicuro in RAM,

incrementando la cima in $O(1)$.

- `stack_pop(ReportStack s)` : Estrae l'ultimo punto di ripristino per innescare l'azione di *Revert* (annullamento modifiche) del cittadino, decrementando la cima in $O(1)$.
-

3. ADT ReportAvl (Albero Auto-Bilanciante Polimorfico)

- **File Header:** `include/adt/report_avl.h`
 - **File Sorgente:** `src/adt/report_avl.c`
 - **Definizione Fisica:** Nodo bilanciato generico (`NodeAVL`) contenente un elemento puntatore opaco `void *elem` , l'indicatore di altezza (`height`), il contatore di sottoalbero (`size`) e il discriminante enumerato `Type_Avl` .
 - **Operazioni Chiave:**
 - `insert(ReportAvl t, void *elem, Type_Avl type, int (*compare)(const void *, const void *))` : Inserimento polimorfico regolato da callback esterne. Gestisce i duplicati logici di User ID facendoli scivolare a destra, attivando le rotazioni LL, RR, LR, RL al rilevamento di un *Balance Factor* fuori dal range $[-1, 1]$.
 - `inorder(ReportAvl t, FILE *file)` : Visita simmetrica profonda che scarica i dati sul disco eliminando ogni spazio spurio, generando gli indici sequenziali ordinati (*Inorder Arrays*) da 21 e 22 byte.
-

4. ADT PriorityQueue (Coda con Ordinamento Incrociato)

- **File Header:** `include/adt/priority_queue.h`
 - **File Sorgente:** `src/adt/priority_queue.c`
 - **Definizione Fisica:** Catena di nodi ordinati in RAM durante la fase di inserimento in base a metriche combinate.
 - **Operazioni Chiave:**
 - `pq_enqueue(PriorityQueue pq, Report r)` : Inserimento a scansione lineare ordinata. Applica la precedenza all'urgenza scalare decrescente ('2' -> '1' -> '0') e, in caso di perfetta parità di urgenza, invoca l'helper `compare_dates_fifo` per dare la precedenza alla segnalazione con la stringa temporale della data più remota (regola FIFO).
 - `pq_dequeue(PriorityQueue pq)` : Estrazione immediata in testa in tempo costante $O(1)$ della segnalazione a massima criticità comunale assoluta.
-

5. Motori d'Indice AVL Persistenti su Disco (File Interrogations)

Per massimizzare l'efficienza ed evitare l'occupazione impropria di memoria in RAM sul server municipale, le funzioni di ricerca per chiave **non lavorano sugli alberi residenti in memoria**, ma interrogano direttamente i file d'indice sequenziali ordinati generati dagli AVL tramite algoritmi di scansione hardware:

- **int findReportId(unsigned int report_id)** : Apre il file `report_AVL_BY_REPORT_ID.txt` e individua l'indice della riga (`disk_row`) del file Master associato al report ricercato. Interroga l'inorder array tramite **ricerca binaria logaritmica su file** in tempo logaritmico certo pari a $O(\log n)$. Poiché l'albero AVL effettua lo scaricamento ordinato dei blocchi a passi fissi da 22 byte (`AVL_REPORT_ID_LINE`), il server legge dal registro comunale il numero complessivo di record censiti ed esegue salti logaritmici mediante combinazioni di `fseek` posizionali sul punto medio (`mid`). Questo azzerla la necessità di effettuare scansioni sequenziali lineari in $O(n)$, garantendo che la ricerca della riga fisica Master rimanga efficiente ed esente dal volume complessivo dei dati stoccati.
- **int findUserId(unsigned int uid, unsigned int **results)** : Apre il file `report_AVL_BY_USER_ID.txt` e isola tutti i Report ID associati all'User ID interrogato. La ricerca sfrutta l'ordinamento nativo dell'Inorder Array generato dall'albero AVL; il server localizza un punto di contatto casuale all'interno del file a passi da 21 byte in tempo $O(\log n)$ tramite **ricerca binaria**. Trovato l'intervallo, un puntatore scansiona a ritroso il blocco per isolare l'indice di partenza esatto, e un ciclo sequenziale estrae i Report ID associati riallocando dinamicamente la memoria RAM tramite raddoppio geometrico predittivo. Questo approccio garantisce la totale scalabilità del sistema, isolando la memoria tramite una doppia referenza ed evitando il degradamento prestazionale dell'I/O sul disco.